

Declarative Explanations for Graph Neural Networks: A Demonstration

Haitong Tang
Zhejiang University
Zhejiang, China
tht@zju.edu.cn

Tingting Zhu
Zhejiang University
Zhejiang, China
tingtingzhu@zju.edu.cn

Yinghui Wu
Case Western Reserve University
Cleveland, USA
yxw1650@case.edu

Tingyang Chen
Zhejiang University
Zhejiang, China
chentyang@zju.edu.cn

Arijit Khan
Bowling Green State University
Ohio, USA
arijitk@bgsu.edu

Xiangyu Ke
Zhejiang University
Zhejiang, China
xiangyu.ke@zju.edu.cn

ABSTRACT

Graph Neural Networks (GNNs) have proven effective for graph analytical tasks, yet their “black-box” nature complicates understanding of their layer-wise behavior and hinders their application in large graphs. We demonstrate SliceGXQ, an end-to-end GNN explanation tool that supports declarative and interactive generation and access of layer-wise explanations for GNNs over large-scale graphs. In particular, SliceGXQ highlights (1) a feature-rich interactive graphical user interface with configurable options (e.g., GNN model selection, query mode, and explanation settings); (2) a lightweight SPARQL syntax parser that allows users to express their query needs for generating and accessing explanations; and (3) a layer-specific explanation generation algorithm that delivers in-depth insights tailored to the user’s query. We showcase the application of SliceGXQ in model debugging and diagnosis, large-scale interpretable network fraud detection, and how this helps users understand the inner workings of GNNs through layer-wise explanations. **Our demonstration video and code are at <https://github.com/Hai0709/SliceGXQ.git>.**

PVLDB Reference Format:

Haitong Tang, Yinghui Wu, Arijit Khan, Tingting Zhu, Tingyang Chen, and Xiangyu Ke. Declarative Explanations for Graph Neural Networks: A Demonstration. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Hai0709/SliceGXQ.git>.

1 INTRODUCTION

Graph Neural Networks (GNNs) have demonstrated remarkable success across various tasks, including node classification. By leveraging message passing and node feature aggregation, GNNs effectively learn node representations by integrating graph structural information with node features. This capability has enabled the

widespread applications of GNNs in domains such as social network analysis, bioinformatics, and recommendation systems.

Due to complex message passing mechanisms and end-to-end learning, GNNs are considered “black-box”, making their decision-making process opaque to users. To address this, recent efforts in graph data management and machine learning [2] have developed various GNN explainer variants (e.g., [4, 6, 7]). These methods identify essential substructures and node features to provide explanations of “why” the GNN model outputs such predictions and “which” components have the most impact on outcomes.

However, existing GNN explanation techniques exhibit several limitations [1, 3]: (1) Most of them provide *static, one-time* explanations that focus solely on the final outputs rather than offering *progressive insights* into how a GNN processes information at each layer. This lack of fine-grained interpretability hinders users from understanding their internal reasoning behavior. (2) Many explainers are themselves “black-box”, making it difficult to validate their outputs; and (3) Users often need to tailor explanations to specific requirements, such as filtering for particular substructures in the output or focusing on specific layers of the model, which typically requires manual intervention and domain expertise. Hence, a *declarative* wrapper is essential for users to efficiently specify queries and easily customize explanations according to their needs.

To this end, we introduce a declarative framework that supports a class of SPARQL-like explanatory queries, called SliceGXqueries, to facilitate layer-wise explanations¹ of GNNs. A SliceGXQ query Q follows a general structured format:

EXPLAIN ($\mathcal{M}$, V_T) **AT** l_t
FROM $\{G, \mathcal{L}\}$
WHERE P , k **WITH** *mode*

Specifically, the following declarative operators are integral to SliceGXQ queries: (1) **EXPLAIN** specifies the primary objective of the query, which is to generate explanations for a pre-trained GNN model ($\mathcal{M}$) and test nodes of interest (V_T); (2) **AT** allows users to inspect specific layer (l_t) within the GNN to interact with, enabling fine-grained, progressive analysis; (3) **FROM** defines the graph dataset (G) and selected layers ($\mathcal{L}$) to generate explanations for model behaviors at l_t ; (4) **WHERE** applies “shape” constraints on the explanations as subgraphs, such as filtering by a *graph/path pattern* (P), as seen in SPARQL queries; or limiting the result size

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

¹We develop layer-wise explanations of GNNs in our full research paper, SliceGX [5].

(k); and (5) **WITH** specifies the explanation mode (e.g., model diagnosis vs. interpretation, as detailed in §3). Additionally, SliceGXQ supports SQL-like operators, such as ORDER BY, which allows users to sort the generated explanations based on specific criteria.

SliceGXQ addresses critical challenges in GNN explanation, including limited granularity, interactivity, and customization by introducing a declarative paradigm that allows users to access layer-wise explanations through SPARQL-like queries. The key contributions of SliceGXQ are summarized as follows.

- To our best knowledge, SliceGXQ is the first system that provides insights into the decision-making process at each layer of the GNNs, and allows users to specify flexible and configurable explanatory queries tailored to application needs.
- SliceGXQ includes an interactive visualization toolkit that facilitates dataset and GNN model configuration, inference-time browsing of explanations, and statistical analysis. This interactive interface empowers users to gain finer-grained layer-wise insights into both data properties and model behavior.
- We demonstrate the effectiveness of SliceGXQ in providing detailed, layer-by-layer insights for model diagnosis through a real-world application. By visualizing how the model processes information at each layer, users can better understand its internal behaviors and refine the model accordingly.

2 SYSTEM OVERVIEW

The workflow of SliceGXQ is given in Figure 1. The system comprises four key front-end modules: Graph Data Selection, GNN Model Selection, Query-Based Explanations, and Visualization Displayer. Among these, the Query-Based Explanations module serves as the central interface for front-end and back-end interactions. The SliceGXQ back-end also consists of four primary modules: Query Interpretation, Model Slicing, Explanation Subgraph Finding, and Verification Operator. These modules process user inputs and configurations received from the front-end, retrieve necessary data from the database, and generate interpretability information for the GNN, which is then returned to the front-end for visual interaction.

Configuration: The Dataset Selection Module enables users to specify the dataset and target test nodes for explanation. The Model Selection Module allows configuration of the GNN architecture. The Visualization Displayer presents corresponding data and model visualizations, and all configurations are subsequently forwarded to the Query-Based Explanation module for explanation generation.

Query-Based Explanations: The core of SliceGXQ is its Query-Based Explanation module, which supports two input modes: Configuration Mode, designed for novice users, and Query Mode, intended for advanced users. While the former supports easy configuration through a user-friendly web form, the latter enables more flexible and detailed analysis by allowing customized parameters and diverse query patterns through SPARQL-like statements in the query interface. Specific examples of such statements are shown in Figure 2. The SliceGXQ system then parses the input statements and passes them to the back-end, where four modules work sequentially to implement layer-wise, progressive interpretations.

(1) **Query Interpretation** module transforms input SPARQL-like queries into executable tasks for explainability. It enables dynamic exploration of configurations and explanations, tailored to user

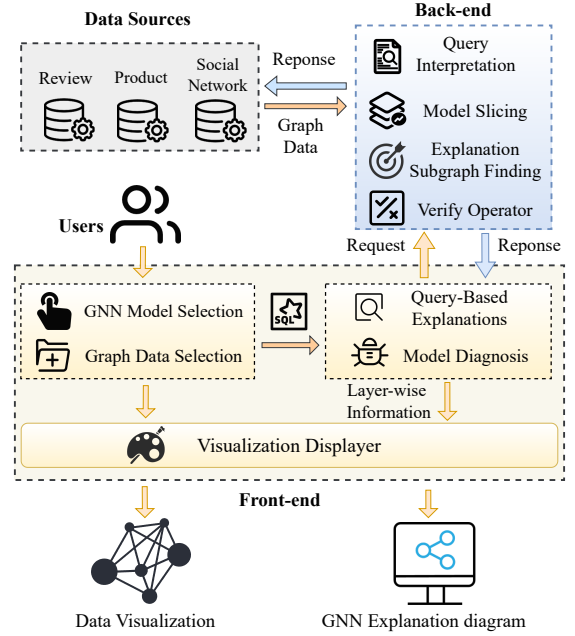


Figure 1: SliceGXQ comprises a front-end and a back-end. Users can select a dataset and model on the front-end, input a query, and the back-end will execute the corresponding algorithm to generate GNN explanation results.

needs. The module ensures that queries adhere to the graph schema and are error-free; otherwise, it returns an error message. Once validated, the query is translated into a configuration C that specifies $(G, \mathcal{M}, V_T, \mathcal{L}, l_t, k)$, where G represents the input graph, $\mathcal{M}$ denotes the GNN model, V_T is the set of target nodes, $\mathcal{L}$ refers to a set of source layers in $\mathcal{M}$, l_t is target layer to be explained, and k specifies a size bound w.r.t. the number of explanatory nodes. The configurations are constructed by a parser from user inputs (in “Configuration Mode”) and passed on to the explanation procedures.

(2) **Model Slicing** module takes the original GNN model as input and segments it into smaller, interpretable local sub-models, each corresponding to a specific layer. These localized sub-models form the foundation for generating layer-specific explanations, allowing finer-grained analysis of each layer to model output.

(3) **Explanation Subgraph Finding** employs a greedy strategy to identify high-quality nodes and generates an optimal explanation subgraph for a particular GNN layer. The input includes the original graph, model slices, and configuration parameters, while the output is a subgraph that satisfies explainability and constraint requirements. The purpose is to determine key nodes and their connections to help explain the GNN model’s prediction logic.

(4) **Verify Operator** module ensures that the explanation subgraph satisfies both counterfactual and factual properties (defined in §3) for the original GNN model and the sliced model. The input of this module comprises the original graph, the target layer, and the candidate explanation subgraph, while the output is a validated subgraph that clarifies the factors influencing the model’s decisions in the target layer. Its key function is to verify the counterfactual and factual relevance of the subgraph to the model’s predictions.

Q_1	EXPLAIN($M, \{v\}$) At 3 FROM $\{G, \{1\}\}$ WHERE 4
Q_2	EXPLAIN($M, \{v\}$) At 3 FROM $\{G, \emptyset\}$ WITH diagnose
Q_3	EXPLAIN($M, \{v_1, v_2\}$) At 3 FROM $\{G, \emptyset\}$ WITH interpret
Q_4	EXPLAIN($M, \{v\}$) At 3 FROM $\{G, \{1, 2\}\}$ WHERE $\langle \text{'Fraud'}, \text{'-'}, \text{'Fraud'} \rangle$
Q_5	EXPLAIN($M, \{v_1, v_2\}$) At 3 FROM $\{G, \{1, 2\}\}$ ORDER BY node.id ASC

Figure 2: An example of SPARQL-like statements.

Model Diagnose: This module is designed to improve GNN model performance through GNN explanations. It identifies potential weaknesses within a specific layer in a GNN and enhances the model’s accuracy and effectiveness. Layer-specific techniques used in this process include (1) layer fine-tuning, i.e., refining the parameters of a specific GNN layer while retaining all original layers; (2) architecture optimization, that is, prune or simplify the specific layer to optimize the model architecture.

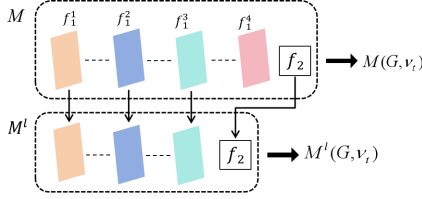


Figure 3: An example of a sliced model for $l=3$

3 CORE ALGORITHM

We next introduce the key techniques in SliceGXQ, which enable the generation of connected explanation subgraphs at specific layers with flexible configurations (see details in [5]). Given a pre-trained GNN model M , a graph G , and a target node v_t with an output $M(G, v_t)$ to be explained, our explanations consist of a set of *explanatory nodes* that are important for clarifying the output $M(G, v_t)$ at an “earlier” stage of inference of GNN M and a *connector set* V_c that ensures the connectivity of the generated subgraphs.

Model Slicing. Given a GNN M with a stack of an L -layered encoder f_1 and a predictor f_2 , and a selected layer $l \in [1, L]$, the l -sliced model of M , denoted as M^l , is constructed by (1) a feature extractor, comprising the first l GNN layers in encoder f_1 that generate output embeddings up to layer l ; (2) the predictor, which applies the model predictor f_2 in M to obtain the classification result through node embeddings. Figure 3 illustrates the construction of an l -sliced model, where only the first l layers of the encoder are retained to generate embeddings, followed by the predictor for classification. Notice that model slicing poses challenges such as the need for additional retraining and potential inconsistencies in model behavior, which can compromise the integrity of its end-to-end inference. The verify operator below effectively addresses these issues by providing reliable explanations based on factual and counterfactual properties, all without requiring retraining.

Verify Operator module ensures that the connected subgraphs adhere to factual and counterfactual properties of both the original GNN model and the sliced model. Specifically, for a target layer l , an explanation subgraph G_s^l (a subgraph of G) satisfies at least one of

the following: (1) [Factual] $M(G, v_t) = M(G_s^l, v_t)$ and $M(G, v_t) = M^l(G, v_t) = M^l(G_s^l, v_t)$; (2) [Counterfactual] $M(G_s, v_t) = M(G \setminus G_s^l, v_t)$ and $M(G, v_t) = M^l(G, v_t) \neq M^l(G \setminus G_s^l, v_t)$.

Explanation Generation employs a greedy strategy to select high-quality nodes to construct an optimal explanation subgraph for $M(G, v_t)$ at each specific layer. To evaluate the quality of a subgraph, we introduce a bi-criteria explainability score that combines *relative influence* and *embedding diversity*, prioritizing explanations that accumulate greater influence and reveal more diverse nodes in terms of node embedding. Given a pre-trained GNN model M and a target layer l , the *explainability* of an explanation subgraph G_s^l , with the explanatory node set V_s^l , at layer l is defined as:

$$f(G_s^l) = \gamma I(V_s^l) + (1 - \gamma) D(V_s^l) \quad (1)$$

where (1) $I(V_s^l)$ quantifies the relative influence of V_s^l ; (2) $D(G_s^l)$ measures the embedding diversity of V_s^l ; and (3) $\gamma \in [0, 1]$ is a balance factor between them. Note that $f(G_s^l) \in [0, 1]$. While alternative measures exist, they are often tailored to final-layer predictions and overlook the distinct characteristics of intermediate representations in GNNs. To capture the model’s behavior at each layer, our bi-criteria explainability score is specifically designed to address these challenges by focusing on intermediate-layer representations, which enables a more nuanced understanding of how intermediate features contribute to the model’s decision-making process.

This process requires generating all explanation subgraphs sequentially for each test node and each GNN layer. To deal with multiple test nodes or deep GNN models, we introduce two optimization techniques for incrementally generating explanations [5].

Diagnose vs. Interpret Mode extends explanations to support two distinct scenarios, each targeting different objectives in understanding and analyzing GNNs: (1) **Diagnose Mode** focuses on identifying the key components responsible for “incorrect” results in a specific layer when misclassification occurs in model M ; and (2) **Interpret Mode** aims to understand the internal inference process of GNNs. We extend the notion of explanation with model-slicing to these two scenarios, with their applications in [5].

4 DEMONSTRATION SCENARIO

In this section, we present the workflow of SliceGXQ and demonstrate its advantages through model debugging. These scenarios highlight the practical value and versatility of SliceGXQ in addressing real-world challenges and optimizing system performance.

Scenario 1: Walkthrough of Workflow: SliceGXQ is designed to facilitate a streamlined, three-step process to enhance user experience and operational efficiency. **Step 1: Dataset Selection** - Users begin by choosing a dataset from the drop-down list, as shown in the upper left corner of Figure 4. They can then visualize the types and quantities of nodes and edges using pie and bar charts, respectively. Nodes of interest can be added to the collection by specifying their identifiers or directly clicking on the graph. **Step 2: Model Selection** - Users configure GNN architectures by adding or removing layers with different functionalities and can view prediction results in a paginated tabular format. **Step 3: SliceGXQ Query** - Users can choose between Configuration Mode and Query Mode for input. Configuration Mode is designed for novice users

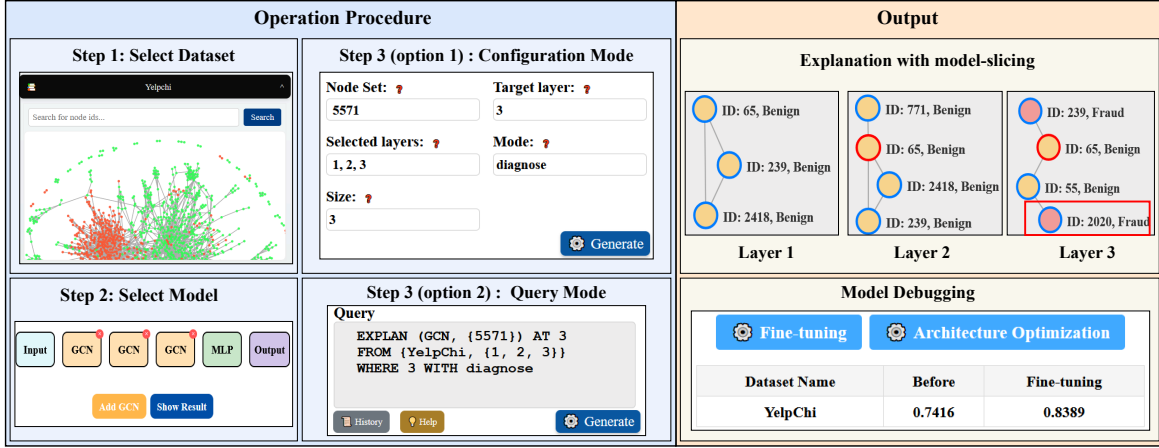


Figure 4: The SliceGXQ system consists of three parts: the dataset module, the model module, and SliceGXQ inference module. In the output panel, nodes in red (resp. yellow) refers to those with label “Fraud” (resp. “Benign”); nodes with blue border (resp. red border) are the explanation nodes (resp. connector nodes).

who are unfamiliar with SliceGXQ, while Query Mode is tailored for experts proficient in SPARQL-like SliceGXQ.

Scenario 2: Model Diagnosis: To demonstrate the functionality of SliceGXQ, consider an example where node 239 (ground-truth label “Benign”) is misclassified as “Fraud” by GCN $\mathcal{M}$. After the user submits a query through the input interface (Figure 4), SliceGXQ sequentially invokes four modules — Query Interpretation, Model Slicing, Explanation Subgraph Finding, and Verify Operator to generate the explanation. The resulting GNN explanations are visualized in a layer-wise manner as labeled subgraphs extracted from the original graph, highlighting the most influential nodes and edges that contribute to the model’s decision. The explanations of layers 1 to 3 reveal that $\mathcal{M}$ performs correctly up to layer 2, suggesting that the misclassification likely arises at hop 3, where node 2020 (labeled “Fraud”) introduces inference bias.

Scenario 3: Large-scale Interpretation: To evaluate the effectiveness and scalability of SliceGXQ, we tested it on large benchmark graph datasets: **Coauthor CS (CS)** (18 333 nodes, 163 788 edges) and **Amazon (AM)** (1 569 960 nodes, 264 339 468 edges). Figure 5 illustrates that as the number of layers increases, fidelity also improves, reflecting a greater effectiveness in progressive explanation generation, while there is a modest increase in running times to generate explanations. This demonstrates that SliceGXQ scales well on large graphs, consistently generating high-quality explanations across different layers. SliceGXQ also outperforms existing GNNExplainer [4] and SubgraphX [7] methods. In Figure 5, the results for GNNExplainer and SubgraphX are shown as individual data points at layer 3 (i.e., maximum number of GNN layers used), since they only produce one-time explanations for the final GNN output. Due to limited space, more empirical analyses are given in [5].

Scenario 4: Model Debugging: Model debugging is facilitated through *fine-tuning*, which focuses on refining a specific GNN layer identified by SliceGXQ as contributing most to classification errors. For example, on the YelpChi dataset, fine-tuning the third layer improves the model’s accuracy by 12%, as shown in Figure 4. These approaches enhance model performance and efficiency through

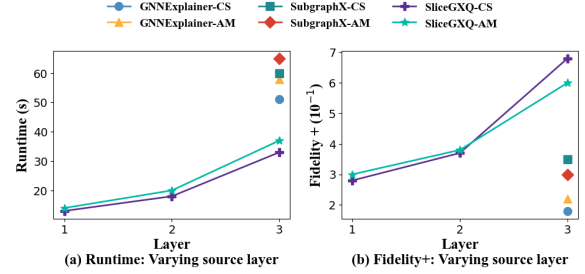


Figure 5: SliceGXQ: Progressive explanation’s efficiency and effectiveness vs. number of GNN layers

precise layer-wise optimization via declarative operations on the layers of GNNs. More empirical analyses are provided in [5].

5 CONCLUSION

We develop SliceGXQ, a user-friendly, interactive and declarative platform for accessing configurable, queryable GNN explanations via model slicing and a class of structured explanatory query operators. Our demonstration provides human experts with a comprehensive visual, operational tool that enables exploration of explanations in depth, facilitating an in-depth understanding of the model’s internal behavior for model interpretation and diagnosis demands. Additionally, it allows users to configure and customize explanations, making it adaptable across diverse application scenarios.

REFERENCES

- [1] T. Chen, D. Qiu, Y. Wu, A. Khan, X. Ke, and Y. Gao. 2024. View-based Explanations for Graph Neural Networks. *Proc. ACM Manag. Data* 2, 1 (2024), 40:1–40:27.
- [2] A. Khan, X. Ke, and Y. Wu. 2025. Graph Data Management and Graph Machine Learning: Synergies and Opportunities. *SIGMOD Rec.* 54, 2 (2025), 28–42.
- [3] A. Khan, X. Ke, Y. Wu, and F. Bonchi. 2026. GNN Explainers 2.0: User-centric and Data-driven Insights. In *WSDM*. 1359–1362.
- [4] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. In *NeurIPS*.
- [5] C. Yu, T. Zhu, T. Chen, Y. Wu, A. Khan, and X. Ke. 2026. SliceGX: Layer-wise GNN Explanation with Model-slicing. In *The Web Conference*.
- [6] H. Yuan, J. Tang, X. Hu, and S. Ji. 2020. XGNN: Towards model-level explanations of graph neural networks. In *KDD*.
- [7] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *ICML*.